

Multi – level Linked List – Print List(Main List and Child)
–Space Complexity and Algorithm

Program:

```
void printList(Node *node, int level = 0)
{
    while (node != nullptr)
    {
        for (int i = 0; i < level; i++)
            cout << "    ";
        cout << "|-- " << node->data << endl;

        if (node->child != nullptr)
        {
            printList(node->child, level + 1);
        }
        node = node->next;
    }
}

...

case 9:
    cout << "Current List Structure: " << endl;
    printList(head);
    break;
```

Space Complexity – Analysis

1. Input Space Complexity

Input space complexity refers to the memory occupied by the inputs to the algorithm.

A. Parameter – Based Input Space (Shallow)

This looks only at the variables explicitly passed into the function per call.

- *The function takes two parameters: a pointer (Node * node) and an integer (int level).*
- *Regardless of the size of the linked list, passing a pointer and an integer takes a fixed amount of memory (typically 4 or 8 bytes for a pointer, and 4 bytes for an int).*
- ***Complexity:*** $O(1)$ *per function call.*

B. Full Data / Holistic Input Space (Deep)

This looks at the actual memory footprint of the underlying data structure being passed into the algorithm.

- *The function operates on a multi – level linked list. Every node in this structure contains data, a next pointer, and a child pointer.*
 - *If there are N total nodes spread across all levels, the data structure itself occupies memory directly proportional to N .*
 - ***Complexity:*** $O(N)$
-

Auxiliary Space

Imbalanced (Flat)[Best Case Scenario]

<i>Recursion Step</i>	<i>No. of problems</i>	<i>Problem Size</i>	<i>Node's Level (k)</i>	<i>Work /Cost Per Problem</i> ($c_1 \times k + c_2$)	<i>Work done at Level</i>
0	1	$n - 0 = n$	0	$c_1(0) + c_2$	c_2
1	1	$n - 1$	0	$c_1(0) + c_2$	c_2
2	1	$n - 2$	0	$c_1(0) + c_2$	c_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$n - 1$	1	$n - (n - 1)$ $=$ 1	0	$c_1(0) + c_2$	c_2
n (Base Case)	1	$n - n = 0$ (Base Case)	<i>N/A</i>	$T(0) = 1$	$1 \times T(0)$ $= T(0)$

Here we see ,Node's level `k` are always 0 i.e. Child $\rightarrow$ Always 0 , and next increases ,but no recursive call stack takes place as, child always remains `0` .

```

if (node->child != nullptr)
{
    printList(node->child, level + 1);
}

```

i. e. node $\rightarrow$ child = nullptr ; always remains ``nullptr``.

1. Input Space Complexity

- **Holistic (The entire data structure): $O(N)$**

Even though the list is flat, if there are N nodes connected via next pointers, the memory required to store the actual linked list in the computer's memory remains proportional to N .

- **Parametric (The function arguments): $O(1)$**

The initial call to `printList(Node * node, int level = 0)` simply passes a pointer and an integer. This takes a constant amount of space regardless of the list's size.

2. Auxiliary Space Complexity: $O(1)$

*Because the condition `if (node  $\rightarrow$  child != nullptr)` is always false, **no recursive calls are ever made.***

Without recursion, no new frames are pushed onto the call stack.

*The function completes its work entirely within the first call stack frame using the iterative while loop. Since it only allocates a few local variables (`node`, `level`, `i`) that do not scale with the size of the input, the extra, temporary memory used during execution is strictly **constant** ($O(1)$).*

Imbalanced Deep[Worst Case Scenario]

<i>Step/ Depth(k)</i>	<i>Nodes Remaining (i)</i>	<i>Node's Level (d_i)</i>	<i>Time in for loop $c_1 \times d_i$</i>	<i>Constant Work (c₂)</i>	<i>Total Work ($c_1 \times d_i$ + c₂)</i>	<i>Current Equation</i>
1	N	0	$c_1 \times 0 = 0$	c₂	0 + c₂	$T(N) =$ $T(N - 1) + c_2$
2	N - 1	1	$c_1 \times 1 = c_1$	c₂	c₁ + c₂	$T(N - 1) =$ $T(N - 2) + (c_1$ $+ c_2)$
3	N - 2	2	$c_1 \times 2 = 2c_1$	c₂	2c₁ + c₂	$T(N - 2) =$ $T(N - 3)$ $+ (2c_1 + c_2)$
....						
N	1	N - 1	$c_1 \times (N - 1)$	c₂	c₁ × (N - 1) + c₂	$T(1) =$ $T(0) +$ $(c_1 \times (N - 1)$ $+ c_2)$
Base Case	0	N/A	N/A	c₂	0 + c₂ = c₂	$T(0) = c$

1. Input Space Complexity

Input space complexity measures the memory required to store the inputs given to the algorithm.

- **Parametric Input Space:** $O(1)$

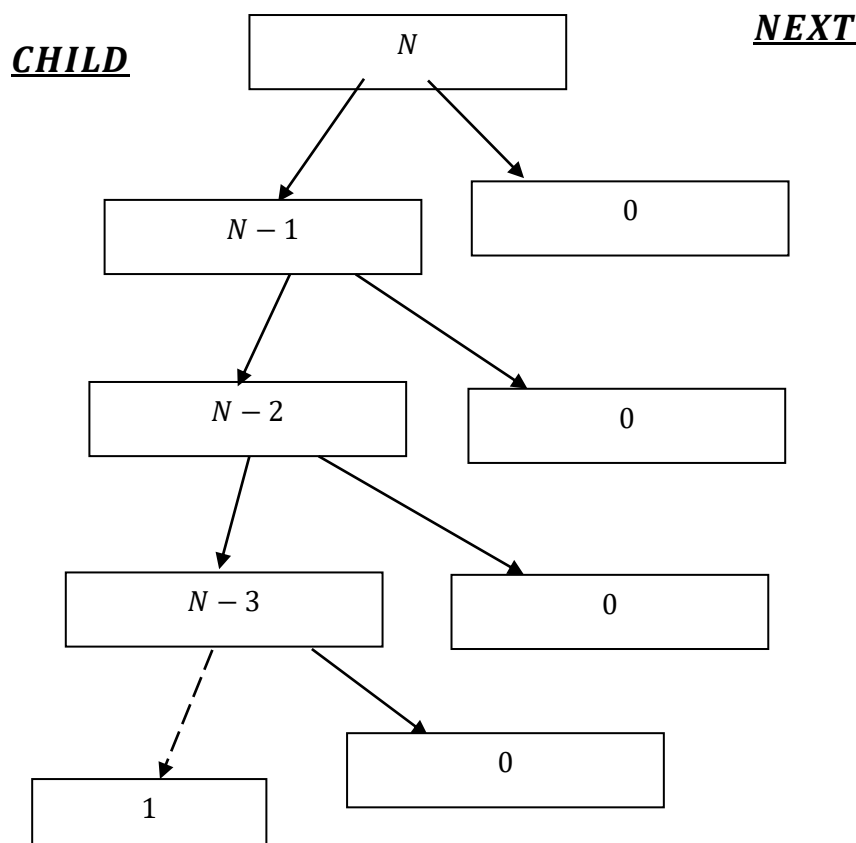
Every time the `printList(Node *node, int level = 0)` function is invoked, it passes two parameters: a pointer to a `Node` and an integer `level`. Regardless of whether the list has 10 nodes or 10,000 nodes, the memory required to store these specific arguments for a single function call remains strictly constant.

- **Holistic Input Space:** $O(N)$

If we consider the total memory footprint of the data structure itself, let N be the total number of nodes in the multi-level linked list. Because each node must be stored in memory (holding data, a next pointer, and a child pointer), the holistic input space is directly proportional to the total number of nodes.

2. Auxiliary Space Complexity

Auxiliary space complexity measures the temporary, extra space used by the algorithm during execution. For recursive functions like this one, it is dictated almost entirely by the **maximum depth of the call stack**.



1. *The root has a problem size of N .*
2. *It recursively calls its child, leaving $N - 1$ nodes.*
3. *That node recursively calls its child, leaving $N - 2$ nodes.*
4. *This continues down a single vertical chain.*

Because the recursion is forced to go N levels deep before it hits a base case (a node with no child), the system must keep N function calls open on the call stack simultaneously. Therefore, the auxiliary space complexity is $O(N)$.

Balanced Tree [Worst Case Scenario]

$$2T\left(\frac{N-1}{2}\right) + c_1 \times k + c_2, \text{ Base Case: } T(0) = 1.$$

<i>Recursion Step</i>	<i>No. of problems</i>	<i>Problem Size</i>	<i>Node's Level (k)</i>	<i>Work /Cost Per Problem</i> ($c_1 \times k + c_2$)	<i>Work done at Level</i>
0	1	n	0	$c_1(0) + c_2$	c_2
1	2	$\frac{n-1}{2}$	1	$c_1(1) + c_2$	$2 \times (c_1 + c_2)$ $= 2c_1 + 2c_2$
2	4	$\frac{n-3}{4}$	2	$c_1(2) + c_2$	$4 \times (2c_1 + c_2)$ $= 8c_1 + 4c_2$
3	8	$\frac{n-7}{8}$	3	$c_1(3) + c_2$	$8 \times (3c_1 + c_2)$ $= 24c_1 + 8c_2$
⋮	⋮	⋮	⋮	⋮	⋮
k	2^k	$\frac{N - (2^k - 1)}{2^k}$	k	$c_1(k) + c_2$	$2^k \times (c_1 \cdot k + c_2)$
$\log_2(N+1)$ (Base Case)	$N+1$	$\frac{N - (2^{\log_2(N+1)} - 1)}{2^{\log_2(N+1)}}$ $= 0$ (Base Case)	N/A	$T(0) = 1$	$(N+1) \times T(0)$

1. Input Space Complexity

The input space remains consistent with the previous scenarios, as the fundamental properties of how the data is passed and stored do not change.

- **Parametric Input Space:** $O(1)$

Each time the `printList(Node *node, int level = 0)` function is called, it allocates memory for two variables: a Node pointer and an integer level. Because this memory requirement per call is constant and does not scale with N , it is $O(1)$.

- **Holistic Input Space:** $O(N)$

Even though the structure is balanced, the data structure still contains a total of N nodes. Storing the entire multi-level linked list in memory requires space directly proportional to the total number of nodes.

2. Auxiliary Space Complexity: $O(\log N)$

The auxiliary space complexity is determined by the **maximum depth of the recursive call stack**.

In recursive algorithms traversing a tree, the call stack does not hold all nodes at once; it only holds the nodes along a single path from the root to the deepest leaf. The maximum number of simultaneous open function calls is equal to the height of the tree.

The final row of the table indicates the Base Case is reached at step

$$\log_2(N + 1).$$

Because the problem size is roughly halved at each vertical step (from n to $\frac{n-1}{2}$, etc.), the recursion only needs to go down $\log_2(N)$ levels before hitting a nullptr child.

Therefore, the maximum depth of the call stack at any given time is $\log_2(N + 1)$. Dropping the constants for Big-O notation, the auxiliary space complexity for this balanced tree scenario is $O(\log N)$.

Algorithm

FUNCTION printlist(NODE, SET LEVEL := 0)

Repeat while NODE ≠ NULL :

// 1. Print indentation based on the current level

For i from 0 to (Level – 1) Do:

Write `` ``

End For

// 2. Print the node's data

Write `` | – – `` , ITEM[NODE] .

// 3. Traverse the child list if it exists

If CHILD[NODE] ≠ NULL Then:

Call printlist(NODE, SET LEVEL := LEVEL + 1)

END If

// 4. Move to the next sibling node

SET NODE := LINK[NODE] .

END WHILE

END FUNCTION